

# Les pools de thread (épisode I)



Fabrice.Kordon@lip6.fr





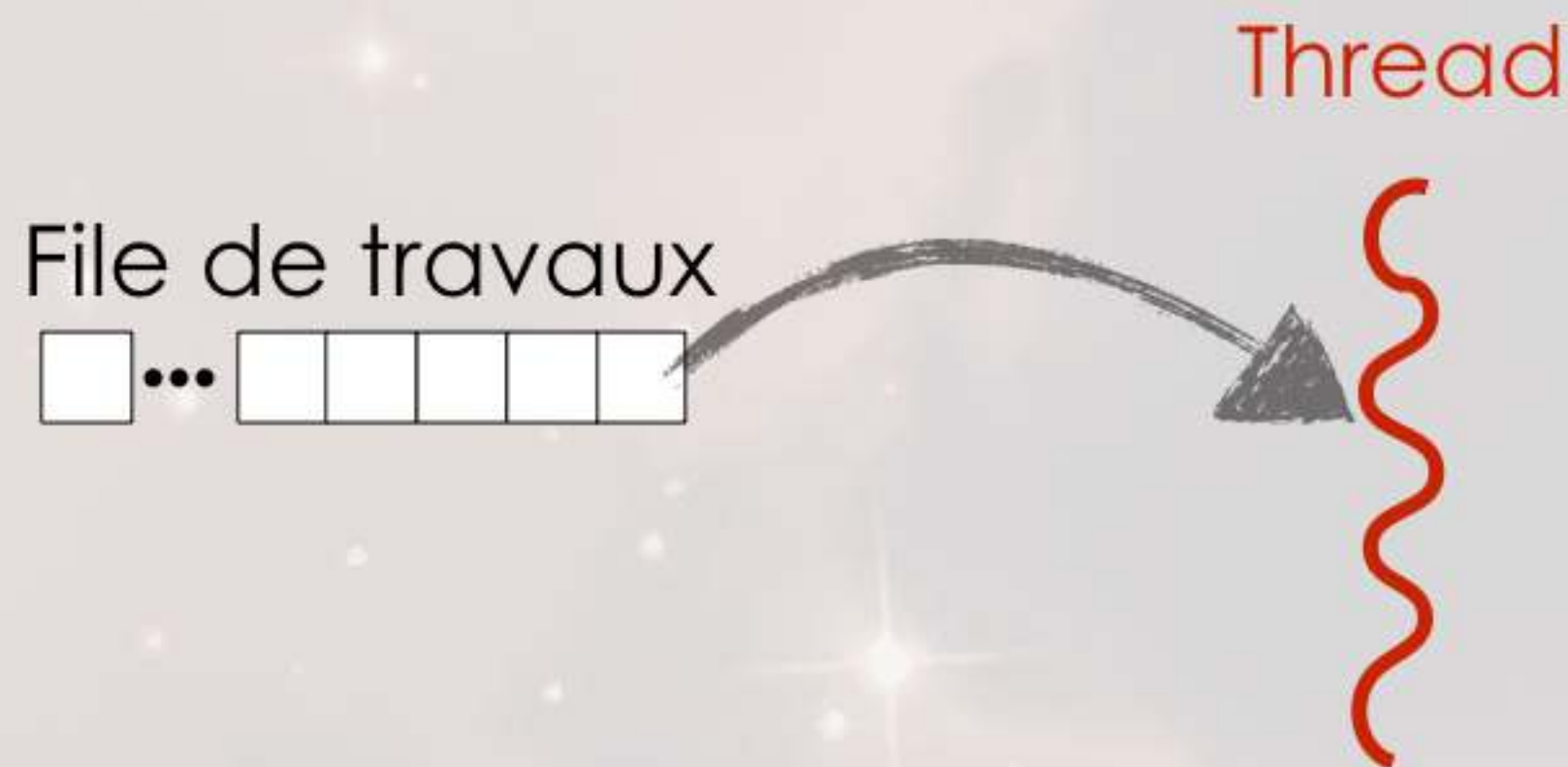
# Qu'est-ce qu'un pool de threads

2

## Pool = réserve

- Disposer d'une réserve de thread pour effectuer des traitements
- Borner (éventuellement en Java) le nombre de thread simultanées

## Principe de fonctionnement





# Qu'est-ce qu'un pool de threads

2

## Pool = réserve

- Disposer d'une réserve de thread pour effectuer des traitements
- Borner (éventuellement en Java) le nombre de thread simultanées

## Principe de fonctionnement

File de travaux



Thread



Réserve de thread





# Qu'est-ce qu'un pool de threads

2

## Pool = réserve

- Disposer d'une réserve de thread pour effectuer des tâches
- Borner (éventuellement en Java) le nombre de threads



### Important

pool = contrôle du grain de parallélisme

## Principe de fonctionnement

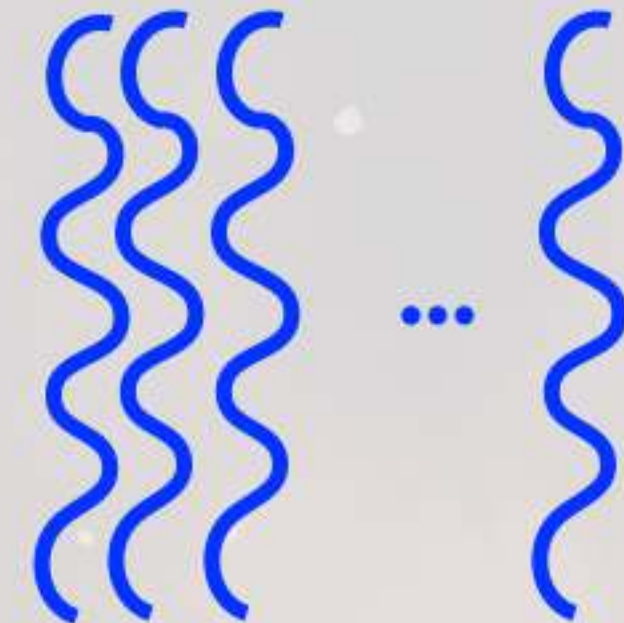
File de travaux



Thread



Réserve de thread



## Quelques caractéristiques

- Pool borné ou non borné
- Recyclage des threads
- Facilité dans la gestion de la terminaison



# L'interface ExecutorService

3

## Base du système de gestion des Pools de Thread

## Quelques services

 `void shutdown()`

- ▶ Ne plus accepter de nouvelles demandes
- ▶ Laisser les tâches non commencées se terminer
- ▶ Arrêter l'exécuteur

 `List<Runnable> shutdownNow()`

- ▶ Forcer l'arrêt immédiat de toutes les threads
- ▶ Rend la liste des tâches dont l'exécution n'a pas commencée

 `boolean awaitTermination(long timeout, TimeUnit unit)`

- ▶ true si toutes les tâches se terminent avant expiration du délai

 `boolean isShutdown()`

- ▶ true si l'exécuteur a été arrêté (primitive shutdown)

 `boolean isTerminated()`

- ▶ true si toutes les tâches sont terminées suite à un shutdown





# La classe Executors

4

## Implémentation concrète des interfaces de base

- Basé sur la classe abstraite `AbstractExecutorService`

## Quelques services

- `static ExecutorService newFixedThreadPool(int nThreads)`
  - ▶ Crée un pool réutilisant un nombre fixé de threads
- `static ExecutorService newCachedThreadPool()`
  - ▶ Crée un pool de taille arbitraire
  - ▶ Réutilisation des threads déjà existentes si possible
- `static ExecutorService newSingleThreadExecutor()`
  - ▶ Pool composé d'une seule thread
- `static ExecutorService newWorkStealingPool()`  
`static ExecutorService newWorkStealingPool(int parallelism)`
  - ▶ Pool garantissant le maximum (ou la valeur fixée) de parallélisme
  - ▶ Garantie que l'on ne dépasse pas `Runtime.availableProcessors()`



# La classe Executors

4



**Attention à l'ordonnancement**

pas forcément de garantie d'équité



**Impléme**



Basé sur la c



**Quelques**



`static ExecutorService newFixedThreadPool(int nThreads)`

► Crée un pool réutilisant un nombre fixé de threads



`static ExecutorService newCachedThreadPool()`

► Crée un pool de taille arbitraire

► Réutilisation des threads déjà existentes si possible



`static ExecutorService newSingleThreadExecutor()`

► Pool composé d'une seule thread



`static ExecutorService newWorkStealingPool()`

`static ExecutorService newWorkStealingPool(int parallelism)`

► Pool garantissant le maximum (ou la valeur fiée) de parallélisme

► Garantie que l'on ne dépasse pas `Runtime.availableProcessors()`



# En guise de conclusion...

5

## Extrêmement pratique

- Rend inutile la gestion explicite d'un ensemble de threads
- Permet de contrôler le nombre de threads
- Implémentation pratique en Java

## Pool de threads $\neq$ mécanisme Java

- Dans les «utilitaires pour la concurrence»
- Implémentation sous Unix
  - ▶ **libpthread**
- Implémentations en C++
  - ▶ À partir de C++'11